



北京師範大學
BEIJING NORMAL UNIVERSITY

计算机图形学 实验报告

姓名 杨博文 学号 202311081015
院系 人工智能学院 专业 计算机科学与技术

2025 年 5 月 30 日

摘 要

计算机图形学的第八次实验，主要内容包括为质点-弹簧系统。

目录

1 运行环境及运行方式	3
2 任务 1:3d 版本	3
2.1 任务要求	3
2.2 数值积分方法	3
2.3 时间与收敛	3
2.4 受力分析	3
3 任务 2:2d 版本	7
3.1 任务要求	7
3.2 数值积分方法	7
3.3 时间与迭代	7
3.4 受力分析	8
4 任务 3: 改变方法	10
4.1 任务要求	10
4.2 三种方法的更新逻辑	10
4.2.1 显式欧拉法	10
4.2.2 隐式欧拉法	10
4.2.3 半隐式欧拉法	11
4.3 时间与收敛	12
4.4 方法比较	12
4.4.1 实现复杂度	13
4.4.2 运行效率	13
4.4.3 显示效果	13
4.4.4 综合结果	13
4.5 结果图片	14

1 运行环境及运行方式

硬件为 Macbook Air, M2 芯片, 操作系统为 MacOS Sequoia 15.0。

IDE 为 Pycharm2024.1.3。虚拟环境由 conda 管理。

2 任务 1:3d 版本

2.1 任务要求

在 “mass_spring_3d.py” 的基础上, 修改质点网格大小 n 、模拟速度 N , 并运行代码。(至少设置三个不同尺度的网格大小, 如 $n=8, 32, 128$) 结合理论课第 13 讲, 分析代码中的 substep 函数中每一帧的更新逻辑, 在实验文档中回答并解释:

- (1) “mass_spring_3d.py” 所用的数值积分方法是哪种?
- (2) 时间步长较大 (N 较小) 时, 该方法是否能收敛? (无法收敛表现为窗口闪烁)
- (3) 如果无法收敛, 测试并给出不同质点网格大小 n 对应的最大时间步长 dt 的估计值。
- (4) 结合代码, 在任一渲染图像上画图分析任一质点的受力情况 (不需要标注具体数值, 弹力画一个即可)。

2.2 数值积分方法

```
v[i] += gravity * dt
v[i] += force * dt
x[i] += dt * v[i]
```

由上述代码可知, 使用的是显式欧拉法。

当前时刻的速度 v 用于预测下一时刻的位置 x , 而当前的位置和速度则用来更新 v 。

2.3 时间与收敛

当 N 较小、 Δt 较大时, 不收敛。下给出测试得出的收敛边界 N 值, 如表 1。

观察到, N 随 n 增加, 先递减再递增, 猜测可能是对勾型函数, 拟合结果为 $N = 0.42n + \frac{680}{n-5} + 53$ 。拟合效果如图 1。 Δt 可由 $\Delta t = \frac{4 \times 10^{-2}}{N}$ 计算得到, 这里不再给出。

2.4 受力分析

在任意时刻, 质点主要受如下几个力: 重力、弹簧内力 (遵守胡克定律, 恢复原状)、弹簧阻尼力 (与质点间相对速度有关, 抵消震荡)、弹簧本身的阻尼、碰撞反作用力。实验结果及受力分析如下一堆图。

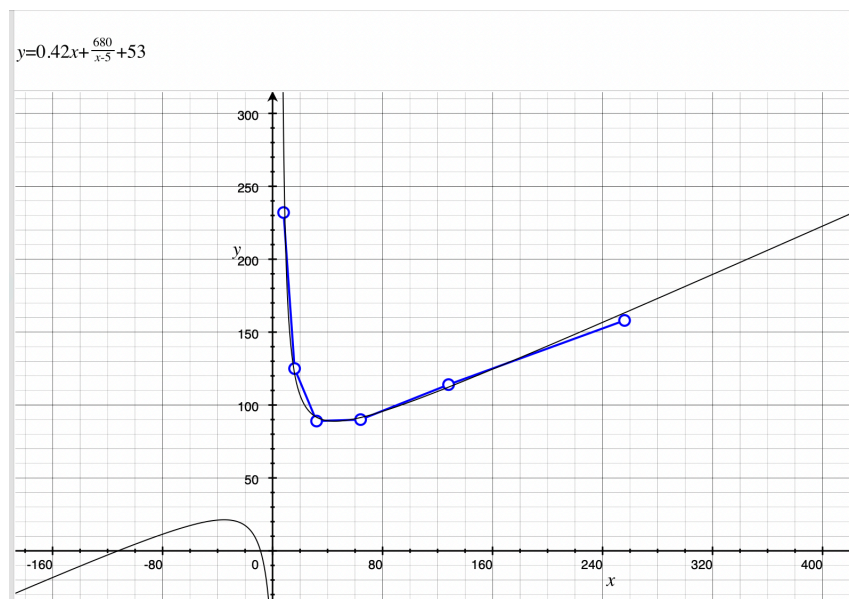
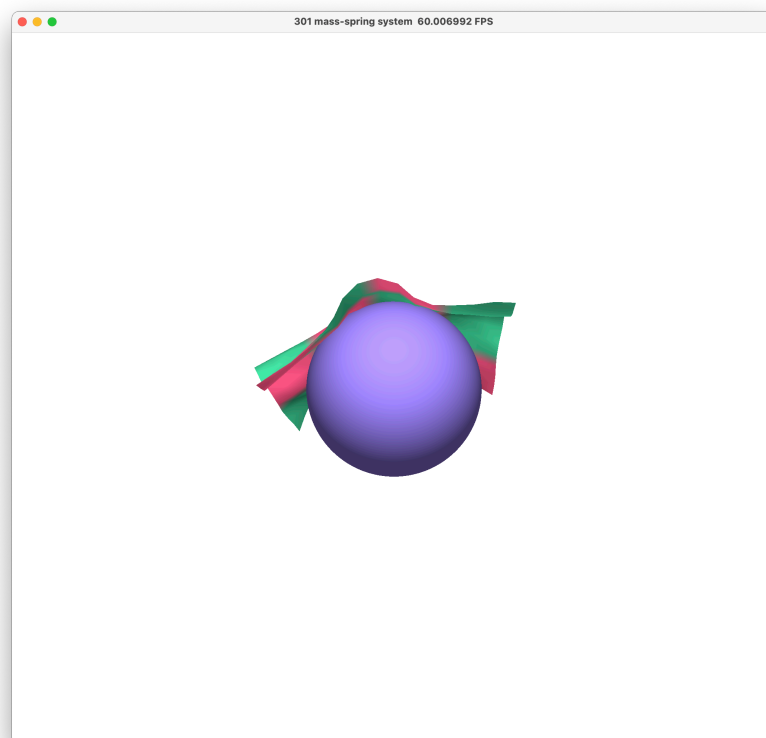


图 1: 实验 1 拟合结果

图 2: 实验 1 成功结果 $n=16$

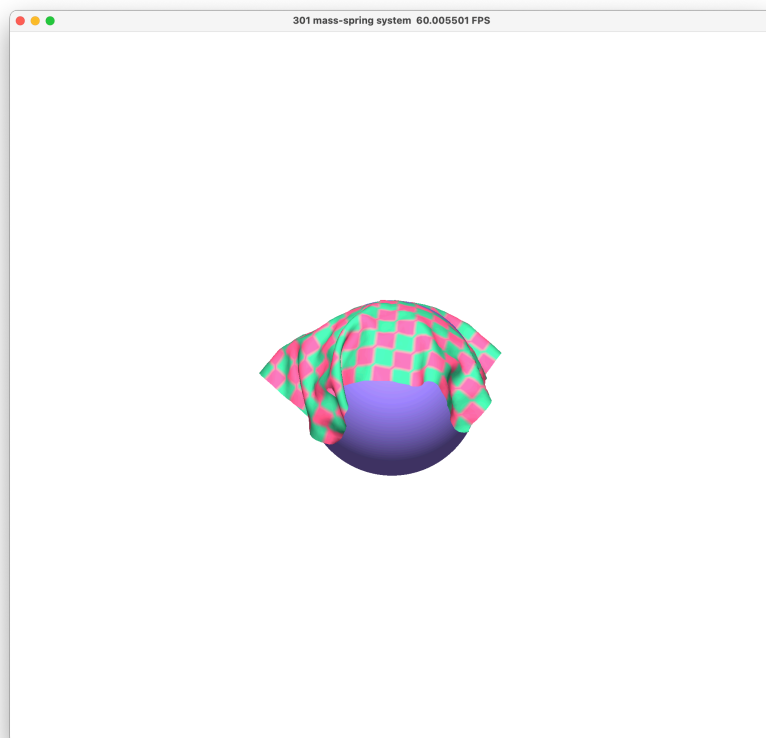


图 3: 实验 1 成功结果 $n=64$

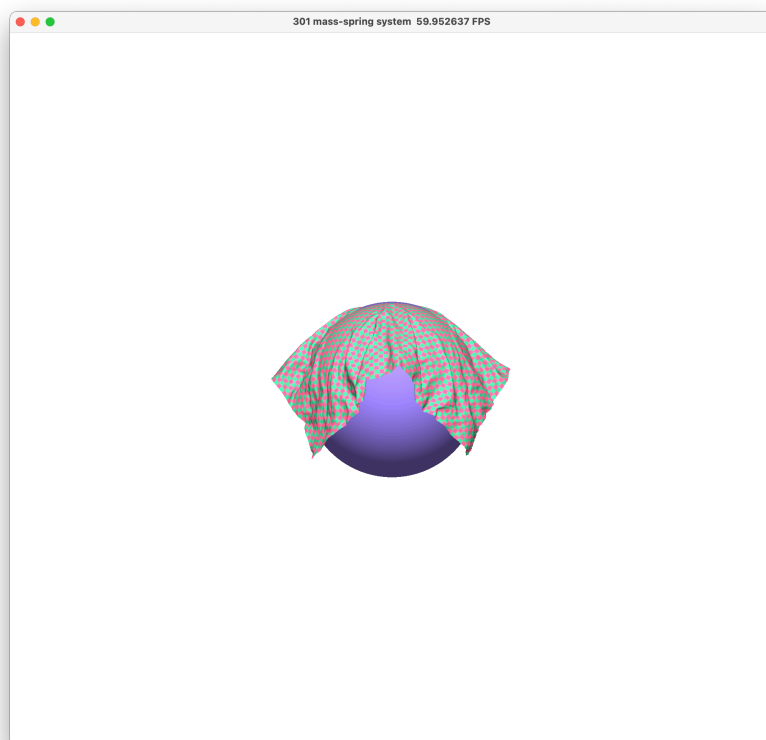


图 4: 实验 1 成功结果 $n=256$

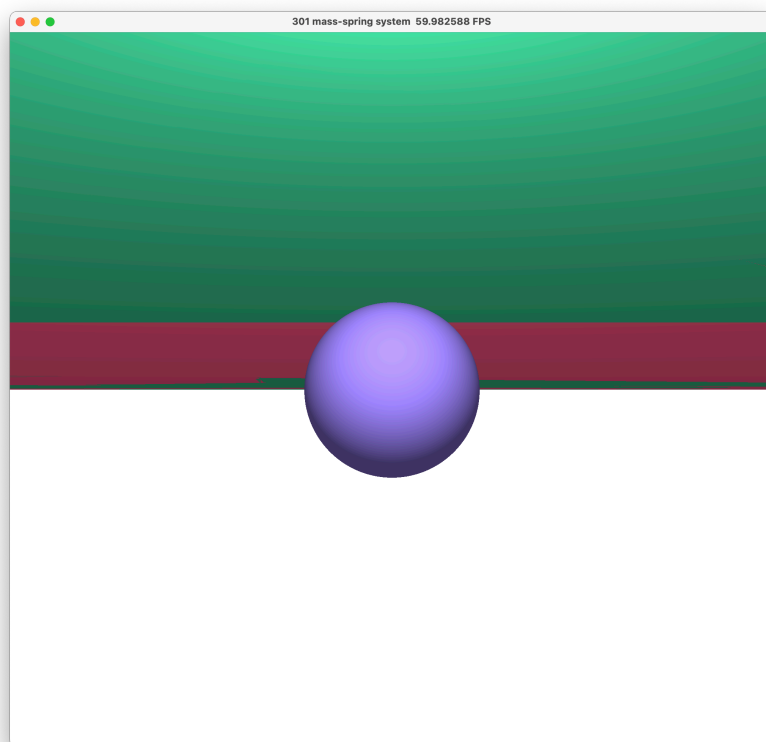


图 5: 实验 1 发散结果

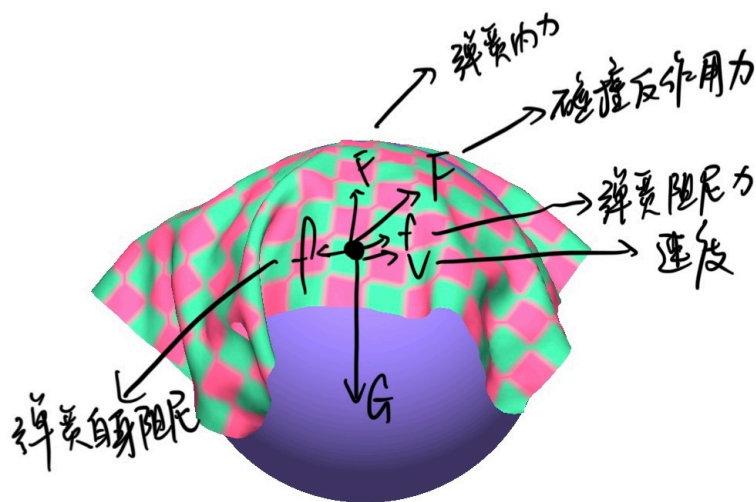


图 6: 实验 1 受力分析

n	N (不闪烁)	$\Delta t = \frac{4 \times 10^{-2}}{N}$
8	232	1.724×10^{-4}
16	125	3.200×10^{-4}
32	89	4.494×10^{-4}
64	90	4.444×10^{-4}
128	114	3.509×10^{-4}
256	158	2.532×10^{-4}

表 1: 不同 n 下对应的不闪烁临界步数 N 及其对应时间步长 Δt

3 任务 2:2d 版本

3.1 任务要求

在“mass_spring_2d.py”的基础上，修改质点网格大小 N 、时间步长 h ，并运行代码。（至少设置三个不同尺度的网格大小，如 $n=8, 32, 128$ ）结合理论课第 13 讲，分析代码中的 `update` 函数及相关子函数中每一帧的更新逻辑，在实验文档中回答并解释：

- (1) “mass_spring_2d.py”所用的数值积分方法是哪种？
- (2) 时间步长较大（ h 较大）时，该方法是否能收敛？（无法收敛表现为窗口闪烁）
- (3) 如果无法收敛，测试并给出不同质点网格大小 n 对应的最大时间步长 dt 的估计值。
- (4) 结合 `compute_force` 函数，在任一渲染图像上画图分析任一质点的受力情况（不需要标注具体数值，弹力画一个即可）。

3.2 数值积分方法

```
# 稀疏求解器，求解Ax=b中的x，即速度增量dv
solver = ti.linalg.SparseSolver(solver_type="LDLT")
solver.analyze_pattern(A) # 分析矩阵模式
solver.factorize(A) # 分解矩阵
# 求解线性系统
dv = solver.solve(self.b)
```

这里使用的是隐式欧拉法。如以上代码所示，其求解了 $A \cdot dv = b$ 这个方程，得到速度增量。所以是隐式欧拉法

3.3 时间与迭代

由于隐式欧拉法相较显式欧拉法更稳定，在多次尝试中并未发现发散情况，系统可以正常运行。

在运行时，总有奇怪的线出现在左下角，原因未知但我认为应当不属于发散情况，原因是在 $n=4$ 时即使 $h=0.000001$ 依然存在这些线。

3.4 受力分析

在任意时刻，质点主要受如下几个力：重力 $F=mg$ 、弹簧内力 $F=-k*x$ 、固定点约束力（左上与右上两个固定点）、阻尼力（体现在雅可比矩阵中）。实验结果及受力分析如下两图。

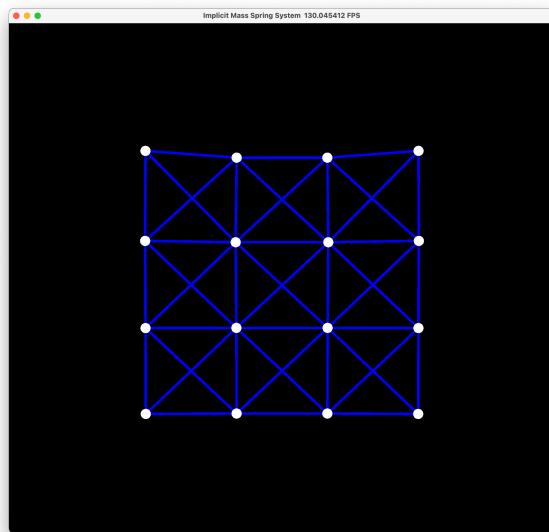


图 7: 实验 2 结果 $N=3$

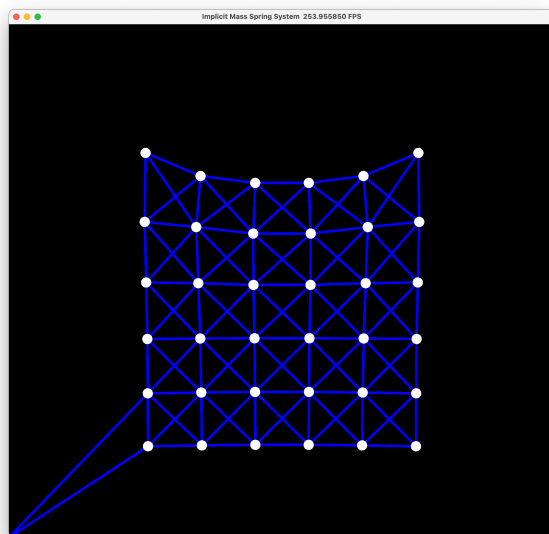


图 8: 实验 2 结果 $N=5$

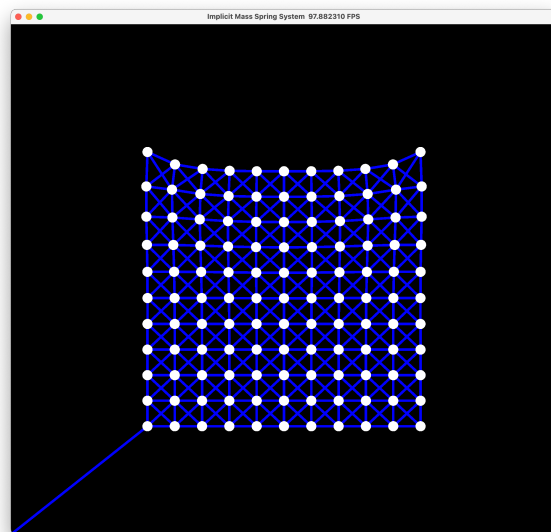
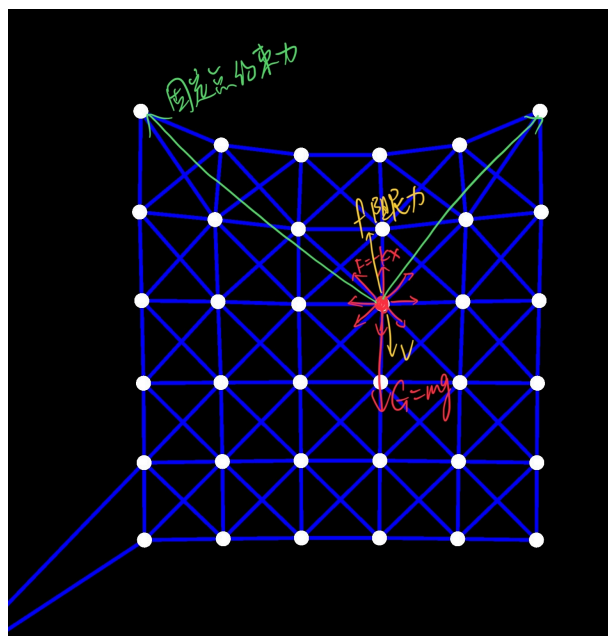
图 9: 实验 2 结果 $N=10$ 

图 10: 实验 2 受力分析

4 任务 3: 改变方法

4.1 任务要求

在“mass_spring_2d.py”的基础上，修改 update 及相应函数，实现数值积分方法（显式欧拉法，隐式欧拉法，半隐式欧拉法）中的另外两种；结合理论课第 13 讲，对比三种数值积分方法的运行结果，保存三种方法的在线渲染帧率、离线渲染图像序列或录制视频，并在实验文档中：

(1) 结合公式与代码，对比关键代码，分析三种数值积分方法的更新逻辑。

(2) 测试并给出对于相同质点网格大小 n ，三种方法下对应的最大时间步长 dt 的估计值。

(3) 对比三种方法的实现复杂度、运行效率、模拟效果；结合运行结果，分析三种方法的优缺点。

4.2 三种方法的更新逻辑

4.2.1 显式欧拉法

```
# 显式欧拉更新位置和速度
@ti.kernel
def updatePosVel(self, h: ti.f32):
    for i in self.pos:
        self.pos[i] += self.vel[i] * h # 先更新位置，这时使用旧的速度
        self.vel[i] += (self.force[i] / self.mass[i]) * h # 更新速度
```

公式如下：

$$v_{new} = v_{old} + \frac{F}{m} \cdot h$$

$$x_{new} = x_{old} + v_{old} \cdot h$$

主要更新逻辑是直接使用旧的速度来计算新的位移。所以说，我们需要先进行位置的更新，然后再更新速度。

4.2.2 隐式欧拉法

```
def update(self, h):
    self.compute_force() # 计算受力

    ### 根据上一时刻的受力、速度和位置计算当前时刻每个质点的速度与位置
    self.compute_Jacobians() # 计算雅可比矩阵
```

```

# 组装全局系统
self.assemble_D(self.DBuilder) # 组装阻尼矩阵
D = self.DBuilder.build()
self.assemble_K(self.KBuilder) # 组装刚度矩阵
K = self.KBuilder.build()
A = self.M - h * D - h**2 * K # 构造系统矩阵

self.copy_to(self.vel_1D, self.vel) # 拷贝速度到一维数组
self.copy_to(self.force_1D, self.force) # 拷贝受力到一维数组

# b = (force + h * K @ vel) * h
Kv = K @ self.vel_1D # 计算Kv
self.compute_b(self.b, self.force_1D, Kv, h) # 计算右端项b

# 稀疏求解器, 求解Ax=b中的x, 即速度增量dv
solver = ti.linalg.SparseSolver(solver_type="LDLT")
solver.analyze_pattern(A) # 分析矩阵模式
solver.factorize(A) # 分解矩阵
# 求解线性系统
dv = solver.solve(self.b)
self.updatePosVel(h, dv) # 更新位置和速度

```

公式如下:

$$v_{new} = v_{old} + \frac{F_{new}}{m} \cdot h$$

$$x_{new} = x_{old} + v_{new} \cdot h$$

其更新逻辑极为复杂, 需要求解方程组。

4.2.3 半隐式欧拉法

```

# 半隐式欧拉更新位置和速度
@ti.kernel
def updatePosVel(self, h: ti.f32):
    for i in self.pos:
        # 先更新速度
        self.vel[i] += (self.force[i] / self.mass[i]) * h # 更新速度
        # 使用更新后的速度来更新位置
        self.pos[i] += self.vel[i] * h # 更新位置

```

公式如下：

$$v_{new} = v_{old} + \frac{F}{m} \cdot h$$

$$x_{new} = x_{old} + v_{new} \cdot h$$

主要逻辑是先用旧的位移计算 F ，然后用新的速度计算新的位移，较为简单（与显示欧拉一样），较稳定。

4.3 时间与收敛

以 $N=10$ 的情况为例，给出测试结果表格如表 2。

方法	最大步长 $h = dt$
隐式欧拉法	均可
半隐式欧拉法	0.021
显式欧拉法	0.0003

表 2: 不同方法下的最大步长 $h = dt$

测试结果与我们的预期是一致的，即在稳定性方面，隐式欧拉法 > 半隐式欧拉法 > 显式欧拉法。

4.4 方法比较

需要注意的是，所给代码“mass_spring_2d.py”在最后部分是逻辑上错误的，具体如下：

```
while window.running:
    cloth.update(h)
    current_t += h

    canvas = window.get_canvas()
    cloth.displayGGUI(canvas)
    window.show()

    if(current_t >= 1.5):
        current_t = 0#此处本来没有
        delta_real_t = datetime.now() - real_t
        print(f"Time taken for 1.5s simulation:
        {delta_real_t.total_seconds()} seconds")
        real_t = datetime.now()
```

如上述代码所示，在 `current_t 1.5` 的分支中，是没有对 `current_t` 进行重置的，也就是说一旦模拟实践 1.5s 就会不停的进行输出。此处应当加入重置。

4.4.1 实现复杂度

显式欧拉法：实现简单，更新位置和速度的公式直接，计算量较少，易于实现。代码量较低。

半隐式欧拉法：比显式欧拉法稍微复杂，但在代码量上仍然较低，易于实现。

隐式欧拉法：最复杂，需要求解线性方程组，实现起来复杂度较高，代码长度更长。

4.4.2 运行效率

在运行效率方面，显式欧拉法的表现通常最为出色，尤其在时间步长较小时，计算速度较快。相比之下，隐式欧拉法由于需要额外的大量计算，效率相对较低。下面是不同方法的运行效率（以 1.5 秒仿真为例，单位：秒）：

显式欧拉法：较快，运行时间相对稳定。

半隐式欧拉法：在较小时间步长下，半隐式欧拉法的运行时间略高于显式欧拉法，但仍然保持较高的效率。

隐式欧拉法：由于其较高的计算复杂度（需要求解方程组），运行时间较长，特别是在时间步长较小的情况下。

4.4.3 显示效果

三者显示效果差不多，但显式欧拉法和半隐式欧拉法可能出现不收敛的问题。

4.4.4 综合结果

综合考虑各方法，总结优缺点如表 3

对比项	显式欧拉法	半隐式欧拉法	隐式欧拉法
实现复杂度	简单	较简单	复杂
运行效率 ($h = 0.0003$)	约 11.5 秒	约 12 秒	约 14 秒
运行效率 ($h = 0.02$)	不收敛	约 0.17 秒	约 0.22 秒
稳定性	较差	好	极好
适用场景	要求快速	较通用	大时间步长
优点	简单，速度快	速度快，稳定好	稳定性强
缺点	稳定性差，易不收敛	速度稍慢	复杂，计算量大

表 3: 显式欧拉法、半隐式欧拉法与隐式欧拉法的综合比较

测试结果数据如下：

$h=0.0003$, 显

Time taken for 1.5s simulation: 11.635346 seconds

Time taken for 1.5s simulation: 11.428123 seconds

Time taken for 1.5s simulation: 11.75471 seconds

Time taken for 1.5s simulation: 11.443873 seconds

$h=0.0003$, 半隐

Time taken for 1.5s simulation: 11.922215 seconds

Time taken for 1.5s simulation: 11.12442 seconds

Time taken for 1.5s simulation: 13.487358 seconds

Time taken for 1.5s simulation: 12.542435 seconds

Time taken for 1.5s simulation: 11.406606 seconds

$h=0.0003$, 隐

Time taken for 1.5s simulation: 14.517459 seconds

Time taken for 1.5s simulation: 13.973502 seconds

Time taken for 1.5s simulation: 13.824635 seconds

Time taken for 1.5s simulation: 14.042568 seconds

$h=0.02$, 半隐

Time taken for 1.5s simulation: 0.556821 seconds

Time taken for 1.5s simulation: 0.159781 seconds

Time taken for 1.5s simulation: 0.177918 seconds

Time taken for 1.5s simulation: 0.171982 seconds

Time taken for 1.5s simulation: 0.186704 seconds

$h=0.02$, 隐

Time taken for 1.5s simulation: 0.673178 seconds

Time taken for 1.5s simulation: 0.261857 seconds

Time taken for 1.5s simulation: 0.248226 seconds

Time taken for 1.5s simulation: 0.222523 seconds

Time taken for 1.5s simulation: 0.215614 seconds

4.5 结果图片

实验图片如下几图。

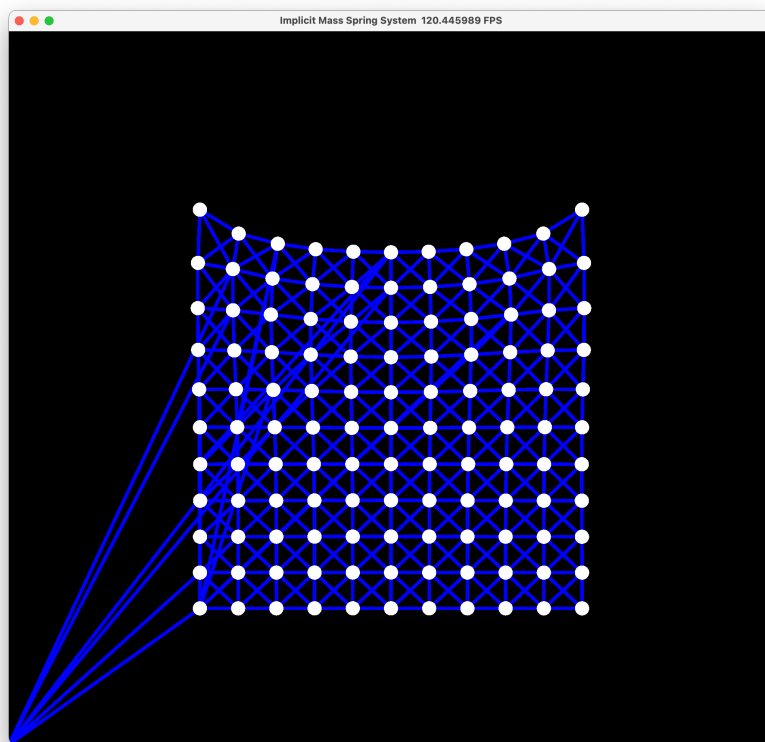


图 11: 实验 3 结果显式欧拉法

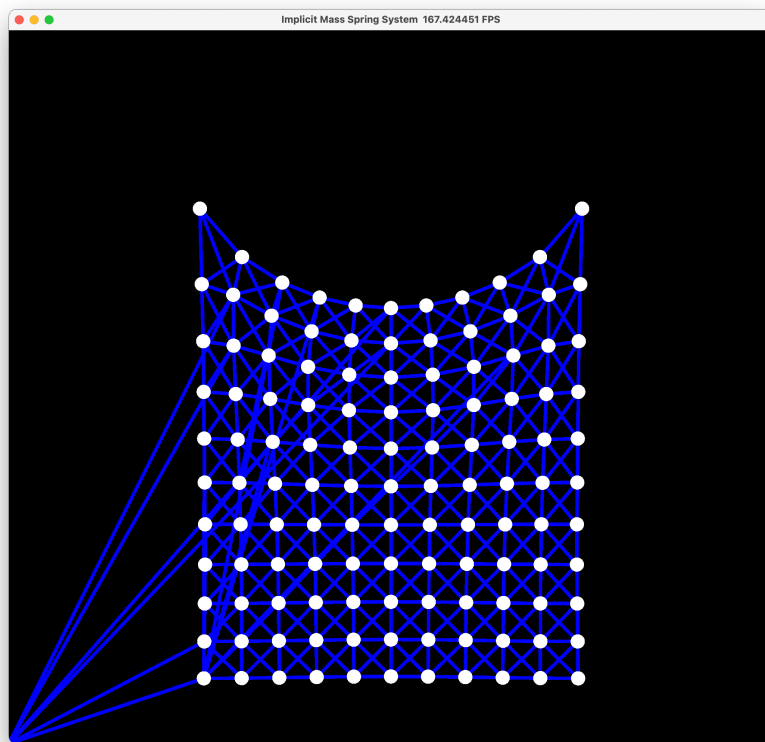


图 12: 实验 3 结果半隐式欧拉法

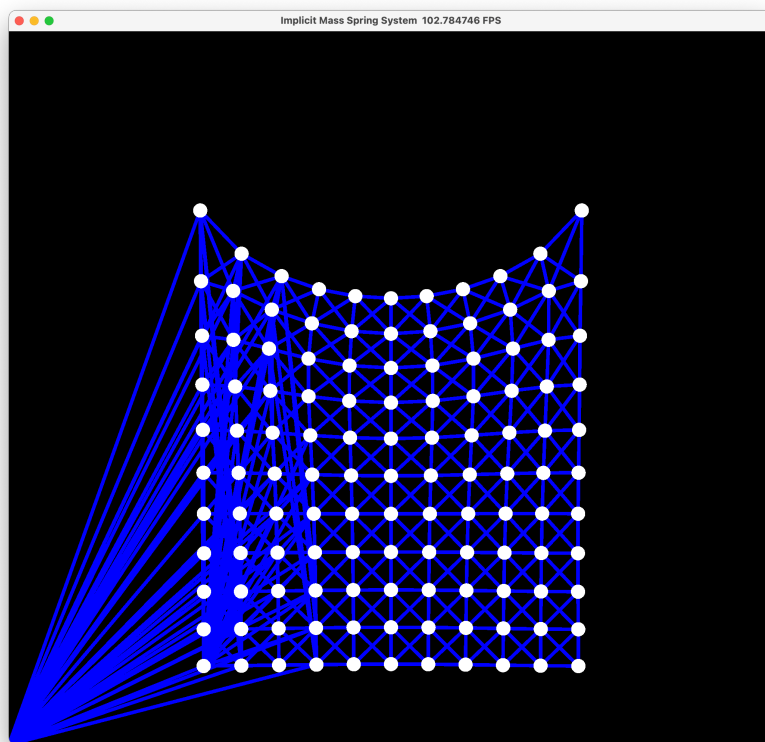


图 13: 实验 3 结果隐式欧拉法

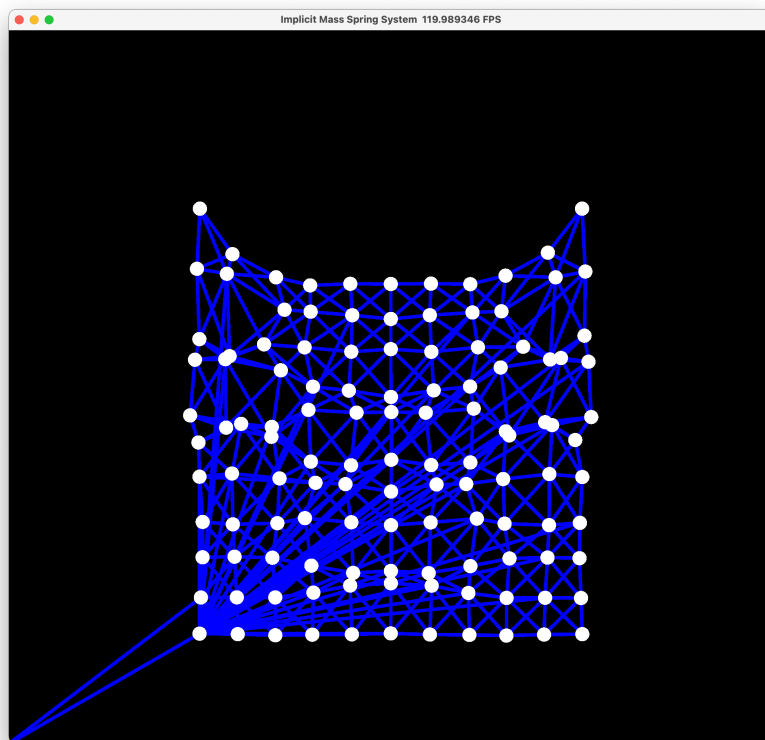


图 14: 实验 3 结果显式欧拉法发散

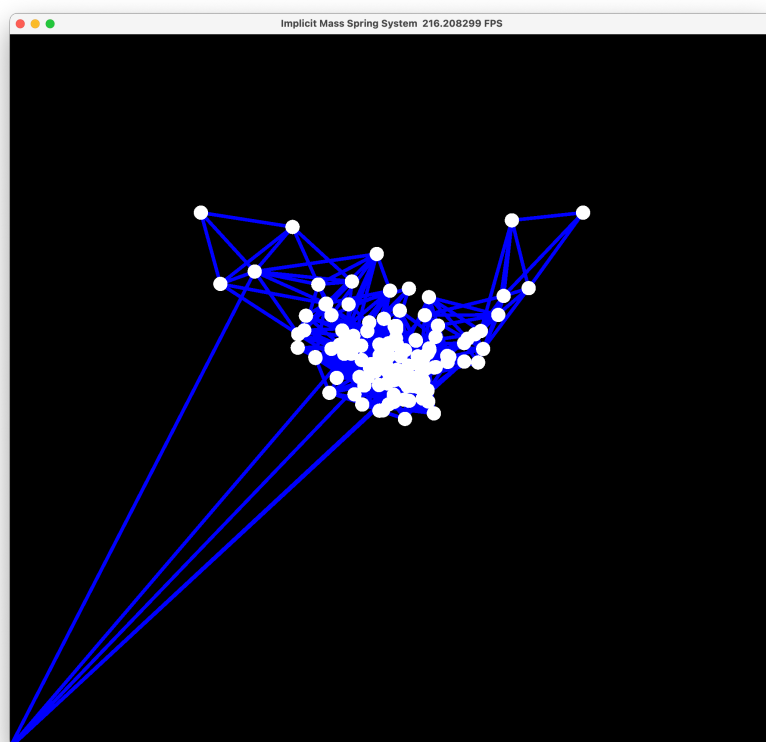


图 15: 实验 3 结果显式欧拉法发散